SweeTy Shop — Website Documentation

Two HTML Files | Demo Mode + Firebase Cloud Mode | GitHub Pages + WhatsApp/Facebook Ordering

SweeTy Shop est une boutique de pâtisseries algérienne moderne, mobile-first, sans build tools, 100% statique compatible GitHub Pages.

🗂️ Fichiers du projet

```
/
├── index.html               → Boutique cliente (storefront)
├── admin.html               → Dashboard administrateur complet
├── firebase-config.json     → (optionnel) Config Firebase pour tous les vis
├── firestore.rules          → Règles Firestore sécurisées
├── storage.rules            → Règles Storage sécurisées
├── assets/                  → Images optionnelles
└── README.md
```

Deux modes :

Mode A — Local Demo Mode (fonctionne immédiatement)

- Données d'exemple (8+ produits, 7 catégories)
- `localStorage` pour sauvegarde locale
- Éditeur produit, thème, homepage, boutique
- Preview instantané
- Export/Import JSON
- Indicateur : `DEMO MODE — DATA IS LOCAL`
- **Attention** : les données locales ne sont visibles que sur ce navigateur, pas publiées sur internet.

Mode B — Firebase Cloud Mode

- Configuration Firebase directement dans `admin.html` → onglet **Firebase Configuration & Cloud Connection**
- Pas besoin d'éditer le code source
- Champs : apiKey, authDomain, projectId, storageBucket, messagingSenderId, appId, measurementId
- Boutons : Save, Test Connection, Connect, Disconnect, Clear, Export, Import, View Status
- Après connexion réussie : `CLOUD MODE — CONNECTED`
- Auth requise pour écriture cloud
- Pour que `index.html` utilise le cloud pour tous les visiteurs : exportez `firebase-config.json` depuis l'admin et uploadez-le à la racine du repo GitHub Pages. `index.html` tente de fetch ce fichier automatiquement, sinon fallback demo/local cache.

🚀 Déploiement GitHub Pages (étape par étape)

1. **Créer repo GitHub** : ex `sweety-shop`, public.
2. **Uploader fichiers** : `index.html`, `admin.html`, `README.md`, (optionnel `firebase-config.json` après config).
3. **Enable Pages** : Settings → Pages → Source: Deploy from branch → Branch: main / root → Save.
4. **Ouvrir site** : `https://votre-user.github.io/sweety-shop/` → boutique cliente.
5. **Ouvrir admin** : `https://votre-user.github.io/sweety-shop/admin.html`
6. **Démo** : tout fonctionne sans Firebase. Ajoutez produits, changez thème, testez WhatsApp.
7. **Créer projet Firebase** : https://console.firebase.google.com → New Project → `sweety-shop-...`
8. **Ajouter Web App** : Project Settings → Your apps → Web → copier config.

9. **Configurer via admin** : dans admin → Firebase tab → coller chaque champ → Save → Test → Connect.

10. **Activer Auth** : Firebase Console → Authentication → Get started → Email/Password → Enable → Add user → `admin@sweety.dz` + mdp fort.

11. **Activer Firestore** : Firestore Database → Create database → Start in test mode (temporaire) → Location `eur3` ou proche.

12. **Appliquer règles sécurisées** : voir section règles ci-dessous → Firestore → Rules → coller `firestore.rules` → Publish.

13. **Configurer Storage (optionnel)** : Storage → Get started → appliquer `storage.rules` → Publish.

14. **WhatsApp & Facebook** : dans admin → Boutique & livraison → renseigner numéro international sans + (ex `213555123456`), Page URL, m.me link → Sauvegarder.

15. **Publier produits** : Mode cloud → Login → Produits → Import Demo Data to Cloud (avec confirmation) ou créer manuellement.

16. **Exporter config pour tous** : Firebase tab → Export Configuration → upload `firebase-config.json` à la racine GitHub repo → commit → tous les visiteurs chargent cloud.

17. **Backups** : Backup tab → Export JSON complet régulièrement. Stockez sur Drive.

Note importante : Firebase config entrée dans dashboard est stockée dans localStorage de ce navigateur. Chaque nouvel appareil devra refaire Save/Connect sauf si vous fournissez `firebase-config.json` dans le repo.

🔐 Firebase Authentication & Sécurité

- Aucun mot de passe codé en dur dans `admin.html`.
- Auth via Firebase Email/Password.
- Login screen s'affiche automatiquement en mode cloud si non connecté.
- Écritures cloud bloquées si `!request.auth` (règles).

- Ne jamais exposer Admin SDK, private keys.
- `firebase-config.json` n'est pas secret mais ne contient pas de droits admin — les droits viennent des Rules.

📊 Structure Firestore (source de vérité en cloud)

Collections :

- `products/{productId}` : {id, name, description, category, price, oldPrice, unit, stock, minQty, availability, featured, bestseller, isNew, images[], variants[], sizes[], order, updatedAt}
- `categories/{categoryId}` : {id, name, desc, icon, color, order, visible}
- `settings/main` : objet unique avec tous les paramètres boutique, homepage, WhatsApp, etc.
- `themes/current` : objet thème + customCss

`index.html` charge `products`, `categories`, `settings/main`, `themes/current` si cloud connecté, sinon local.

Ne jamais mélanger local et cloud sans action explicite utilisateur (Import Demo to Cloud).

🖼️ Images

- **Démo** : File input → conversion base64 → localStorage (limite ~5MB). Avertissement si >900KB.
- **Cloud** : Si Storage configuré + user authentifié, upload vers `products/{timestamp}-{filename}` → URL publique stockée dans Firestore.
- Sinon : autoriser URL image externe.
- Ne jamais stocker grosses images en base64 dans Firestore document.

💬 WhatsApp & Facebook Ordering

Aucun paiement en ligne.

WhatsApp : lien `https://wa.me/PHONE_NUMBER?text=ENCODED_MESSAGE`

Message pré-rempli :

```
Salam SweeTy Shop ! 🍰
Je souhaite commander :
• Produit: xxx
• ID: SW-001
• Quantité: 2 box
• Variante: Chocolat
• Taille: 500g
• Prix affiché: 2800 DZD
• Note: ...
Merci de confirmer...
```

Numéro configurable, stocké sans espaces, sans + dans URL. Bouton désactivable.

Option résumé commande avant ouverture.

Client doit envoyer manuellement.

Facebook Messenger : config Page URL, m.me link. Bouton ouvre Messenger si possible sinon Page. Désactivable indépendamment.

Ne jamais prétendre que l'ouverture de l'app confirme la commande.

🎨 Theme Editor

- Variables CSS : --bg, --bg-soft, --surface, --text, --primary, --secondary, --accent, --border, --radius, etc.
- Live preview, couleurs, fonts, radius, largeur page, spacing, custom CSS avec validation basique.

- Presets : Luxury Chocolate, Pink Pastry, Cream & Gold, Modern Minimal, Dark Luxury.
- Changer thème ne supprime pas produits.

💾 Backup & Restore

- Export all → JSON complet (produits, catégories, settings, theme)
- Import JSON avec validation schéma (vérifie products array)
- Export produits CSV
- Backup local avant toute import destructif (`sweety_backup_before_import`)
- Preview import avant appliquer
- Avertissement écrasement

🧪 Testing Checklist

Voir fichier `TESTING_CHECKLIST.md` ci-dessous.

⚠️ Limitations & Bonnes pratiques

- GitHub Pages sert HTML, Firebase stocke données partagées + auth.
- localStorage limité 5-10MB → éviter images base64 trop grosses.
- En démo, chaque navigateur a ses propres données.
- En cloud, lecture publique, écriture auth uniquement.
- Ne jamais réduire stock automatiquement à l'ouverture WhatsApp — uniquement action manuelle admin.
- Ne pas fabriquer commandes/revenus — dashboard affiche stats réelles.

📞 Support

Si Firebase Rules manquantes/incorrectes → admin affiche erreur au lieu de prétendre sécurisé.
Si storage indisponible → explique limitation et permet URLs.

Fait avec ♥ à Tlemcen.